

Contrat API — SASQuATCH Backend

Document de référence pour le développement du frontend React. À jour des routes MUST HAVE implémentées et testées côté backend. Donne ce document en contexte à ton outil d'IA pour qu'il génère du code cohérent avec la vraie API (au lieu de deviner des routes ou des champs).

Informations générales

- Base URL en développement : `http://localhost:8000`
- Toutes les requêtes (sauf `/auth/login`, `/auth/activate`, `/health`) nécessitent le header `Authorization: Bearer <token>`
- Toutes les erreurs renvoient `{"detail": "message en français"}` avec un code HTTP approprié (400, 401, 403, 404, 409, 500)
- CORS déjà autorisé pour `localhost:3000` et `localhost:5173` (Vite/CRA). Si ton serveur de dev tourne sur un autre port, il faut le signaler au backend pour l'ajouter à la liste blanche.
- Documentation interactive complète et toujours à jour : `http://localhost:8000/docs`

Rôles existants

`student`, `teacher`, `admin` — chaque route ci-dessous précise quel(s) rôle(s) peuvent l'appeler.

Authentification

POST /auth/login

Pas d'auth requise.

Requête :

```
json
{ "email": "string", "password": "string" }
```

Réponse 200 :

```
json
{ "access_token": "string (JWT)", "token_type": "bearer", "role": "student | te
```

Réponse 401 si identifiants invalides ou compte non activé.

Stocker `access_token` côté client (ex: en mémoire ou contexte React, PAS en localStorage si possible pour limiter le risque XSS) et l'envoyer dans le header `Authorization: Bearer <access_token>` pour toutes les requêtes suivantes.

POST /auth/activate

Pas d'auth requise (l'utilisateur n'a pas encore de mot de passe).

Requête :

```
json

{ "activation_token": "string (reçu par e-mail, présent dans le lien)", "new_pa
```

Réponse 200 : `{ "message": "Compte activé avec succès" }` Réponse 404 si token invalide, 400 si expiré (48h de validité).

Administration (rôle admin uniquement)

POST /admin/users

Création d'un compte (étudiant ou enseignant).

Requête :

```
json

{
  "institutional_id": "string",
  "nom": "string",
  "prenom": "string",
  "email": "string (email valide)",
  "role": "student | teacher"
}
```

Réponse 201 :

```
json

{ "id": "uuid", "role": "string", "is_active": false }
```

Note : ne renvoie jamais l'email (anonymisation jusque dans l'API). Le lien d'activation est envoyé par e-mail séparément (pas encore branché côté backend à ce jour -- voir avec l'équipe backend pour le statut).

Réponse 409 si l'email existe déjà.

POST /admin/deanonymize

Désanonymisation d'une contribution. Action sensible, journalisée.

Requête :

```
json

{ "question_id": "uuid", "reason": "string (justification obligatoire)" }
```

Réponse 200 :

```
json

{
  "pseudonym": "string",
  "institutional_id": "string",
  "nom": "string",
  "prenom": "string",
  "email": "string",
  "log_id": "uuid",
  "requested_at": "datetime ISO"
}
```

Sessions de cours

POST /sessions

Rôle `teacher`. Ouvre une nouvelle session de cours.

Requête : `{ "label": "string (ex: Cours Réseaux S6)" }`

Réponse 201 :

```
json

{
  "id": "uuid",
  "label": "string",
  "join_code": "string (6 caractères, ex: H549ME)",
  "started_at": "datetime ISO",
  "is_active": true
}
```

Le `join_code` est à afficher/projeter à l'écran pour les étudiants. Le `id` est nécessaire pour ouvrir le WebSocket (voir plus bas) et pour les routes de modération.

POST /sessions/join

Rôle `student`. Rejoint une session active via son code.

Requête: `{ "join_code": "string" }`

Réponse 200:

```
json
{ "session_id": "uuid", "label": "string", "pseudonym": "string" }
```

Le `pseudonym` est stable pour toute la durée de la session -- à afficher à l'étudiant ("Vous participez en tant que XXXX") et à conserver côté client pour les requêtes suivantes si besoin d'affichage.

Réponse 404 si code invalide/session clôturée, 403 si l'étudiant est banni.

Idempotent : rejoindre plusieurs fois renvoie toujours le même pseudonyme.

POST /sessions/{session_id}/close

Rôle `teacher`, doit être le créateur de la session.

Pas de corps de requête.

Réponse 200:

```
json
{ "id": "uuid", "label": "string", "is_active": false, "ended_at": "datetime ISO"
```

Déclenche un message WebSocket `session_closed` à tous les clients connectés.

POST /sessions/{session_id}/ban

Rôle `teacher`, créateur de la session.

Requête: `{ "pseudonym": "string" }`

Réponse 200: `{ "pseudonym": "string", "is_banned": true }`

Déclenche un message WebSocket `participant_banned`.

POST /sessions/{session_id}/unban

Même format que `/ban`, avec `is_banned: false` en réponse. Déclenche un message WebSocket `participant_unbanned`.

Questions

POST /questions

Rôle `student`, doit avoir rejoint la session au préalable.

Requête :

```
json

{
  "session_id": "uuid",
  "content": "string",
  "parent_id": "uuid | null (optionnel, pour une clarification)"
}
```

Réponse 201 :

```
json

{
  "id": "uuid",
  "session_id": "uuid",
  "pseudonym": "string",
  "parent_id": "uuid | null",
  "content": "string",
  "is_filtered": "boolean",
  "filter_reason": "blacklist | rate_limit | null",
  "satisfaction": "satisfied | unsatisfied | null",
  "submitted_at": "datetime ISO"
}
```

Réponse 403 si non inscrit à la session, si banni, ou si `parent_id` ne correspond pas à une question de l'étudiant lui-même.

Important pour l'UX : si `is_filtered: true`, la question est enregistrée mais ne sera PAS poussée par WebSocket au dashboard enseignant. Décider côté UI si l'étudiant doit être informé ou pas que sa question a été filtrée (sujet ouvert, à trancher en équipe).

Pas encore disponible : route GET pour lister/recharger les questions d'une session (utile à l'ouverture du dashboard, avant que de nouvelles questions n'arrivent par WebSocket). À demander à l'équipe backend si besoin pour l'affichage initial.

GET /questions/sessions/{session_id}

Rôle `teacher`, créateur de la session. Liste les questions déjà soumises -- à appeler à l'ouverture du dashboard, AVANT de connecter le WebSocket, pour afficher l'historique déjà existant.

Paramètre optionnel: `(?include_filtered=true)` pour voir aussi les questions bloquées par la modération automatique (false par défaut).

Réponse 200 :

```
json
{
  "questions": [
    {
      "id": "uuid", "session_id": "uuid", "pseudonym": "string",
      "parent_id": "uuid | null", "content": "string",
      "is_filtered": "boolean", "filter_reason": "string | null",
      "satisfaction": "string | null", "submitted_at": "datetime ISO"
    }
  ],
  "total": "integer"
}
```

Ordre : du plus ancien au plus récent (`submitted_at` croissant).

WebSocket temps réel (dashboard enseignant)

Connexion

```
ws://localhost:8000/ws/sessions/{session_id}?token={jwt_access_token}
```

Rôle `teacher`, doit être le créateur de la session `{session_id}`. Le token passe en paramètre d'URL (limite de l'API WebSocket navigateur, qui ne permet pas d'envoyer de header `Authorization`).

Exemple JS :

```
javascript
```

```

const ws = new WebSocket(`ws://localhost:8000/ws/sessions/${sessionId}?token=${token}`);

ws.onopen = () => console.log("Connecté au dashboard temps réel");

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  switch (data.type) {
    case "new_question":
      // data.question = { id, pseudonym, parent_id, content, submitted_at }
      break;
    case "participant_banned":
      // data.pseudonym
      break;
    case "participant_unbanned":
      // data.pseudonym
      break;
    case "session_closed":
      // désactiver l'interface, informer l'utilisateur
      break;
  }
};

ws.onerror = (e) => console.error("Erreur WebSocket", e);
ws.onclose = () => console.log("Déconnecté");

```

Ce WebSocket ne sert qu'à RECEVOIR des messages du serveur -- aucune action n'est attendue en émission depuis le client sur ce canal.

Limite connue à ne pas perdre de vue : si l'enseignant recharge la page ou se reconnecte, il ne reçoit que les NOUVELLES questions à partir de ce moment -- pas l'historique. Tant que la route GET de relecture (mentionnée ci-dessus) n'existe pas, prévoir de garder l'état en mémoire côté React plutôt que de compter sur un rechargement transparent.

Ce qui n'existe pas encore (à ne pas développer en supposant que ça existe)

- Fil de questions regroupées par pseudonyme (SHOULD HAVE)
- Bouton satisfaction côté étudiant (champ `satisfaction` existe dans le modèle, mais aucune route ne permet encore de le modifier)
- Configuration de la modération depuis une interface admin
- Regroupement thématique par IA (COULD HAVE, prévu en tout dernier)
- Envoi réel d'e-mail d'activation (le token existe en base mais n'est pas encore transmis automatiquement)

Si le frontend a besoin d'une de ces fonctionnalités avant qu'elle soit prête côté backend, en discuter en équipe plutôt que d'improviser un format qui ne collera pas à l'implémentation finale.